

PROGRAMME DE FORMATION · VERSION 3 · 2026-09-13

Spring Boot et Angular : Développer des applications Web avec Spring Boot et Angular

Réalisez un parcours web complet reliant Angular à une API Spring Boot. Vous travaillez contrat HTTP, validation, persistance, autorisations et tests afin d'assembler deux applications qui gèrent aussi les erreurs de bout en bout.

3 jours · 21 heures

Intermédiaire · Hybride

1 672 € HT

Tarif indicatif, dates, disponibilité et modalités contractuelles à confirmer avec le centre.

À qui s'adresse cette formation ?

Développeurs Java ou web souhaitant relier une API Spring Boot à une interface Angular.

Prérequis

Pour suivre cette formation SpringBoot et Angular, une connaissance préalable minimale de Spring Boot et d'Angular est indispensable.

Objectifs pédagogiques

- Mener à bien un projet SpringBoot/Angular
- Sécuriser une API Rest avec un modèle stateless
- Mettre en place la sécurité côté client Angular
- Adopter une approche « Design By Contract »
- Savoir tester séparément le service back-end Spring et le client Angular

Le cas fil rouge

Réaliser une application de suivi de demandes avec API Spring Boot, interface Angular et autorisations vérifiées côté serveur.

Préparation

Java, HTTP et bases TypeScript ; JDK, Node, IDE, base locale et projet de départ compatibles entre eux.

Progression détaillée

Répartition pédagogique proposée sur une base de 7 heures par jour, pauses exclues. À ajuster avec le formateur au positionnement initial. Les approfondissements « Pour aller plus loin » sont sélectionnés par le formateur selon le positionnement et le temps disponible ; ils ne constituent pas des heures supplémentaires garanties.

Jour 1 · 7 heures

1. Définir le contrat entre interface et API

210 minutes · apports, exercice et retour

- Modéliser demandes, utilisateurs et états métier.
- Décrire routes, schémas, codes HTTP et erreurs de validation.
- Séparer DTO, modèle de persistance et contrat exposé.
- Préciser DTO, validation, erreurs, pagination et statuts HTTP dans le contrat ; distinguer entité persistée et ressource exposée.

NOTIONS ET MÉTHODES EN DÉTAIL

- Définir ressources, verbes HTTP, statuts et représentation des erreurs avec des exemples de requêtes.
- Séparer DTO public et entité persistée ; préciser champs obligatoires, pagination et tri autorisés.
- Dessiner états chargement, résultat vide et échec du parcours avant les composants.

Atelier prévu

Écrire le contrat de création et de consultation d'une demande, avec exemples valides et erreurs attendues.

Livrable attendu

Contrat d'API et scénarios de parcours.

Vérification en atelier

Le contrat contient un exemple nominal, une validation refusée et une ressource absente.

Pour aller plus loin · selon positionnement et temps disponible

Prévoir une évolution d'API compatible et formaliser le comportement d'une requête invalide ou concurrente.

2. Construire le service Spring Boot

210 minutes · apports, exercice et retour

- Organiser contrôleurs, services et repositories.
- Valider les entrées et délimiter les transactions.
- Préparer pagination, migrations et gestion centralisée des erreurs.
- Séparer contrôleur, service et dépôt ; définir frontières transactionnelles, contraintes et mapping des réponses.

NOTIONS ET MÉTHODES EN DÉTAIL

- Organiser contrôleur, service et repository ; injecter les dépendances par constructeur.
- Valider les DTO et centraliser les erreurs sans renvoyer pile technique ou données internes.
- Délimiter transaction et contraintes de base ; traduire un conflit métier en réponse cohérente.

Atelier prévu

Implémenter création et liste des demandes, puis vérifier rollback et validation sur une entrée volontairement invalide.

Livrable attendu

API transactionnelle et tests de validation/persistance.

Vérification en atelier

Une requête invalide ne modifie aucune donnée et reçoit un message exploitable.

Pour aller plus loin · selon positionnement et temps disponible

Diagnostiquer un chargement excessif des relations et prévenir une modification concurrente perdue.

Jour 2 · 7 heures

3. Développer le parcours Angular

210 minutes · apports, exercice et retour

- Composer pages, composants et service HTTP typé.
- Relier formulaire, validation et messages d'erreur accessibles.
- Gérer chargement, absence de données et navigation.
- Composer composants, services et formulaires réactifs ; traiter chargement, absence de résultat, erreur et annulation d'une requête.

NOTIONS ET MÉTHODES EN DÉTAIL

- Décomposer page, composants et service HTTP ; typer les réponses et les états de chargement.
- Construire formulaire réactif, validation locale et restitution des erreurs renvoyées par l'API.
- Gérer navigation, abonnement et annulation de requêtes pour éviter un affichage périmé.

Atelier prévu

Construire un formulaire et une liste paginée qui restent compréhensibles lorsque l'API répond lentement ou échoue.

Livrable attendu

Écrans Angular avec états d'interface et gestion de concurrence réseau.

Vérification en atelier

Le formulaire couvre saisie invalide, attente, succès et échec serveur.

Pour aller plus loin · selon positionnement et temps disponible

Empêcher une réponse réseau ancienne de remplacer l'état d'une recherche plus récente.

4. Sécuriser le flux de bout en bout

210 minutes · apports, exercice et retour

- Expliquer OAuth2, OpenID Connect et validation des jetons.
- Contrôler rôle et propriété de la ressource dans l'API.
- Distinguer garde de route, CORS, XSS et protections liées au stockage choisi.
- Distinguer authentification, autorisation, CORS et CSRF ; effectuer les contrôles métier côté serveur.

NOTIONS ET MÉTHODES EN DÉTAIL

- Distinguer authentification, autorisation et masquage de bouton ; vérifier les droits côté serveur.
- Analyser cookie, jeton, CORS et CSRF selon l'architecture retenue pour le cas.
- Tester accès anonyme et accès d'un utilisateur à la ressource d'un autre utilisateur fictif.

Atelier prévu

Tester deux utilisateurs : modifier l'URL d'une demande ne doit jamais permettre l'accès au dossier de l'autre.

Livrable attendu

Matrice d'accès et scénarios de sécurité de bout en bout.

Vérification en atelier

La restriction reste effective quand l'appel HTTP contourne directement l'interface.

Pour aller plus loin · selon positionnement et temps disponible

Construire une matrice de droits et prouver qu'un utilisateur ne peut pas lire ou modifier les données d'un autre.

Jour 3 · 7 heures

5. Tester séparément les deux applications

210 minutes · apports, exercice et retour

- Écrire tests de services et de contrôleurs avec dépendances isolées.
- Tester composants et réponses HTTP simulées côté Angular.
- Vérifier compatibilité du contrat sur succès et erreurs.
- Séparer tests unitaires, intégration HTTP/persistance et parcours UI ; utiliser des données de test maîtrisées.

NOTIONS ET MÉTHODES EN DÉTAIL

- Isoler logique métier avec tests unitaires et tester contrôleurs avec validation et autorisations.
- Simuler réponses HTTP Angular et tester les états du composant sans dépendre du serveur.
- Écrire un scénario d'intégration création-consultation-erreur sur une base de test.

Atelier prévu

Introduire une rupture de contrat, la détecter avec les tests puis corriger le backend ou le client avec justification.

Livrable attendu

Suite de tests par couche et rapport de parcours critiques.

Vérification en atelier

Un changement incompatible du contrat provoque l'échec d'un test identifiable.

Pour aller plus loin · selon positionnement et temps disponible

Vérifier qu'une erreur métier remonte avec un message utile et qu'un échec réseau ne détruit pas la saisie.

6. Assembler et préparer la livraison

210 minutes · apports, exercice et retour

- Séparer configurations locales et paramètres de déploiement.
- Construire les artefacts et documenter les variables nécessaires.
- Observer erreurs, santé du service et limites d'exploitation.
- Gérer configurations par environnement, secrets, migrations et observabilité de l'API.

NOTIONS ET MÉTHODES EN DÉTAIL

- Externaliser URL d'API et configuration d'environnement sans publier de secrets dans le frontend.
- Construire les deux applications et définir ordre de démarrage et contrôle de disponibilité.

- Préparer versionnement du contrat, migrations et procédure de retour à une version compatible.

Atelier prévu

Démontrer le parcours complet, remettre tests et guide de lancement, puis expliquer le traitement d'un incident d'authentification.

Livrable attendu

Dossier de livraison, paramètres d'exploitation et recette de déploiement.

Vérification en atelier

La livraison démarre depuis la procédure et le parcours critique passe sa recette.

Pour aller plus loin · selon positionnement et temps disponible

Préparer un déploiement compatible avec la version précédente du frontend et une procédure de retour arrière.

Évaluation finale

Mise en situation individuelle et restitution commentée, incluses dans la dernière séquence. Durée indicative : 60 minutes.

- Le contrat couvre les erreurs autant que le succès.
- Les permissions sont testées dans l'API.
- Backend et interface disposent de tests exécutables séparément.

Méthodes et ressources

Apports illustrés, exercices progressifs, ateliers sur un cas fil rouge et retour argumenté sur les productions. Les ateliers et l'évaluation font partie des heures indiquées.

Les ateliers décrivent les travaux prévus. Leurs fichiers, corrigés et évaluations en ligne sont à préparer, sauf ressources explicitement disponibles dans le module pilote.

Accès, accompagnement et contact

Pour obtenir plus de détails ou le programme détaillé validé, contactez le centre : 09 54 70 23 80 — contact@beyondexpertise.eu.

Positionnement et vérification des prérequis avant inscription. Délais, calendrier et aménagements à convenir avec le centre. Pour un besoin d'accessibilité, décrivez le besoin pratique sans diagnostic médical.

Programme enrichi proposé, à valider par le formateur avant animation. Les approfondissements sont adaptés au niveau et au temps disponible. Aucune certification professionnelle ni prise en charge CPF ou OPCO n'est garantie. Démarche Qualiopi en cours ; certification non acquise.

Références pour approfondir

- Spring Boot · documentation de référence
- Angular · tests de l'application